

AI NéphroCare

Explication détaillée de chaque figure, diagramme et outil du rapport

Plateforme d'aide à la décision clinique en hémodialyse
Cohorte HD-478 · CHU Hassan II de Fès · 2020–2024

SVM Linéaire · AUC = 0,826 · 478 patients · 95 décès à 1 an

Modèle ML : SVM Linéaire, C=0,01, 32 features (24 cliniques + 8 interactions)
Pipeline : KNNImputer(k=3) → PowerTransformer(Yeo-Johnson) → SVC
Calibration : CalibratedClassifierCV (Platt/sigmoid, cv=5)
Backend : Django REST Framework 4.2, PostgreSQL 16, Redis 7, Celery
LLM : Qwen2.5 :14B via Ollama, ChromaDB (RAG), 7 services Docker

Table des matières

1 Outils et technologies — rôle précis dans le projet	2
2 Chapitre 1 — Introduction	7
3 Chapitre 2 — État de l’art	8
4 Chapitre 3 — Vue d’ensemble	8
5 Chapitre 4 — Pipeline de données	9
5.1 Curation manuelle HD-478	9
5.2 Schéma de base de données	10
5.3 Pipeline LLM de prétraitement	11
5.4 Feature Engineering	12
6 Chapitre 5 — Modélisation ML	13
6.1 Comparaison des modèles	13
6.2 Zones de risque	16
7 Chapitre 6 — Architecture logicielle	17

Outils et technologies — rôle précis dans le projet

Cette section explique **exactement ce que fait chaque outil** dans le projet AI NéphroCare, avec les fichiers et paramètres réels.

Django REST Framework 4.2 — Backend API — 5 applications

Rôle dans le projet : Sert les 40+ endpoints REST qui font fonctionner toute la plateforme. Le backend est organisé en 5 applications Django :

- `users` — gestion des comptes, authentification JWT personnalisée. `CustomTokenSerializer` surcharge `get_token()` pour injecter dans le payload : `id`, `email`, `nom`, `prenom`, `telephone`, `role`.
- `patients` — modèle `Patient` avec 160+ champs répartis en 11 sections JSONB; `save()` auto-génère `id_patient`; `icc_charlson` est un `CharField` (pas `Integer`); pipeline LLM de prétraitement complet dans `views.py` (3800+ lignes).
- `predictions` — charge `mortalite_svm.joblib`, extrait les 32 features via `extract_features_for_patient()`, appelle `pipeline.predict_proba(input_df)[: ,1][0]`, sauve en JSON filesystem via `_save_last_prediction()`.
- `audit` — `AuditLog` avec `action=TextField`; chaque opération sensible y est enregistrée automatiquement.
- `config` — `urls.py` routant 5 préfixes API : `/api/auth/`, `/api/patients/`, `/api/predictions/`, `/api/audit/`.

RBAC : classes `IsSuperAdmin` et `IsAdminOrChefService` dans `users/permissions.py`.

React 18 + Axios — Frontend SPA — 6 pages

Rôle dans le projet : Application monopage (SPA) avec 6 pages principales : Login, Dashboard, Patients, Prétraitement, Modèle IA, Monitor.

- `AuthContext.js` — après login réussi, extrait `{access, refresh}` de la réponse, stocke dans `localStorage`, décode le JWT via `jwtDecode(access)` pour initialiser l'état utilisateur (`id`, `email`, `nom`, `rôle`). Pas de refresh automatique sur 401.
- `axios.js` — intercepteur `REQUEST` uniquement : ajoute `Authorization: Bearer <token>` à chaque requête. Aucun intercepteur `RESPONSE` (pas d'auto-refresh sur expiration).
- `ProtectedRoute` — filtre la navigation selon le rôle RBAC; les sections non autorisées ne sont jamais rendues.
- `Chart.js` — rend les 12 graphiques analytiques (6 patients + 6 modèle IA).

PostgreSQL 16 — Base de données — 9 tables, port 5432

Rôle dans le projet : Stocke les données persistantes dans la base `plateforme_medicale`.

- 9 tables : `users_role`, `users_utilisateur`, `patients_patient`,

patients_preprocessvalidationrequest, patients_dynamiccolumnrequest, patients_patientformtemplate, patients_patientformfield, audit_auditlog, audit_hiddenauditlog.

- patients_patient contient 160+ colonnes dont 11 colonnes JSONB (une par section clinique) + extra_data JSONB pour les colonnes dynamiques importées depuis Excel.
- Index GIN sur les colonnes JSONB pour accélérer les recherches dans les sections cliniques.
- Données persistées via volume Docker postgres_data.

Redis 7 — Broker Celery — port 6379

Rôle dans le projet : Redis est utilisé **exclusivement** comme broker de tâches Celery et backend de résultats :

- CELERY_BROKER_URL = 'redis://redis:6379/0'
- CELERY_RESULT_BACKEND = 'redis://redis:6379/0'

Important : Redis ne stocke *pas* les sessions de prétraitement. Celles-ci sont sauvegardées sous forme de fichiers JSON sur le filesystem du conteneur backend (patients/preprocess_sessions/{id}.json).

Celery Worker — Traitement asynchrone du LLM

Rôle dans le projet : Exécute la tâche `analyze_preprocess_async` définie dans `patients/tasks.py` pour éviter les timeouts HTTP pendant les 5–10 minutes d'inférence Qwen2.5 :14B.

Paramètres de la tâche : `session_id` (UUID hex), `file_path` (chemin temporaire dans `/tmp/preprocess/`), `user_id`, `use_llm` (booléen). Le pipeline exécuté :

1. Lecture du fichier Excel/CSV via Pandas
2. Profilage technique (`_build_technical_profile()`)
3. Chunking sémantique (`_build_preprocess_chunks()`)
4. RAG ChromaDB (`_build_retrieval_context()`)
5. Appel LLM Qwen2.5 :14B (`_call_ollama_qwen_analysis()`)
6. Correction KB médicale (`_run_bio_value_correction_pass()`)
7. Imputation KNN (`_run_model_imputation()`, k=5 ici)
8. Sauvegarde session JSON (status="completed")

Ollama + Qwen2.5 :14B — LLM local — port 11434

Rôle dans le projet : Modèle de langage 14 milliards de paramètres, déployé localement sans API externe, pour analyser les fichiers de données cliniques et proposer des corrections justifiées médicalement.

- Endpoint appelé : POST `{OLLAMA_BASE_URL}/api/generate` (défaut Docker : `http://ollama:11434/api/generate`)
- Entrée : prompt structuré contenant le profil technique du fichier (nb lignes, colonnes, % manquants, valeurs aberrantes) + contexte médical RAG (normes néphrologiques de ChromaDB) + échantillons de données

- Sortie attendue : JSON avec `corrections`, `issues`, `pipeline`, `route`, `confidence_contract`
- Justification du choix : déploiement on-premise (confidentialité des données patients), pas de coût d'API, contrôle total sur le modèle

ChromaDB — Base vectorielle RAG — stockage local

Rôle dans le projet : Stocke et recherche des embeddings de chunks cliniques pour fournir un contexte médical pertinent au LLM (RAG).

- Mode : `PersistentClient(path='/tmp/preprocess/chroma')` (client embarqué, pas un serveur séparé)
- Modèle d'embedding : `nomic-embed-text` via Ollama
- Nommage des collections : hash SHA-1 des métadonnées du fichier (lignes, colonnes, % manquants) pour éviter les doublons
- Fallback : si ChromaDB est indisponible, la fonction `build_medical_rag_context()` utilise `medical_kb.json` (base de connaissances locale) pour fournir les normes néphrologiques (albumine, calcium, PTH, ferritine, etc.)
- Requête : `collection.query(query_embeddings=..., n_results=max_chunks, include=['documents', 'metadatas', 'distances'])`

scikit-learn — Pipeline ML — SVM + calibration

Rôle dans le projet : Implémente le modèle de prédiction de mortalité à 1 an. Pipeline exact :

```
Pipeline(steps=[('imputer', KNNImputer(n_neighbors=3)), ('transformer',
PowerTransformer(method='yeo-johnson')), ('clf', SVC(C=0.01,
kernel='linear', class_weight='balanced', probability=True,
random_state=42))])
```

Enveloppé dans : `CalibratedClassifierCV(base_pipe, method='sigmoid', cv=5)`
= Platt scaling.

- **KNNImputer(k=3)** : impute les valeurs manquantes des 32 features par moyenne des 3 voisins les plus proches dans l'espace des features disponibles. Indispensable car 8/32 features ont des valeurs manquantes dans HD-478 (ex. : `du_residuelle` manque pour 40% des patients).
- **PowerTransformer(Yeo-Johnson)** : normalise la distribution de chaque feature vers une gaussienne, quelle que soit son signe (contrairement à Box-Cox qui exige des valeurs positives). Indispensable pour SVM qui est sensible aux échelles.
- **SVC(C=0,01, linear, balanced)** : marge large (petit C) pour éviter le sur-apprentissage sur les 478 patients; `class_weight=balanced` compense le déséquilibre 383 survivants / 95 décès (80/20).
- **Calibration Platt** : transforme les scores de décision SVC en probabilités calibrées. Validé par le Brier Score = 0,130 (excellent < 0,15).

Sauvegardé dans `predictions/models/mortalite_svm.joblib` (compress=3). Métadonnées dans `mortalite_svm_features.joblib` : `feature_keys` (32), `threshold` (Youden), `metrics` (AUC, PR-AUC, Brier, F1).

SimpleJWT — Authentication — HS256, 8h/1j

Rôle dans le projet : Gère l'émission et la vérification des tokens JWT. Configuration dans `config/settings.py` : `ACCESS_TOKEN_LIFETIME = timedelta(hours=8)`, `REFRESH_TOKEN_LIFETIME = timedelta(days=1)`, `ALGORITHM = 'HS256'`, `AUTH_HEADER_TYPES = ('Bearer',)`.

Payload custom (ajouté par `CustomTokenSerializer.get_token()`) : `id`, `email`, `nom`, `prenom`, `telephone`, `role`. Plus les claims standard SimpleJWT : `user_id`, `exp`, `iat`, `jti`, `token_type`.

Endpoints : POST `/api/auth/login/` et POST `/api/auth/token/refresh/`.

Docker Compose — Infrastructure — 7 services

Rôle dans le projet : Orchestre tous les composants en un seul `docker compose up -build -d`. Les 7 services :

1. **medical_db** — PostgreSQL 16, volume `postgres_data`, healthcheck avant démarrage du backend
2. **medical_redis** — Redis 7, broker Celery
3. **medical_backend** — Django, dépend de db + redis healthy
4. **medical_frontend** — React build statique servi par serve/Nginx
5. **medical_ollama** — Qwen2.5 :14B, port 11434, inférence GPU/CPU
6. **medical_celery_worker** — même image que backend, commande `celery -A config worker`
7. **medical_audit_cleanup** — tâche périodique de nettoyage des vieux logs d'audit

openpyxl — Export Excel coloré

Rôle dans le projet : Génère le fichier Excel exporté après prétraitement LLM avec mise en forme visuelle :

- Feuille *Données* : cellules modifiées par le LLM en jaune pâle (`#FFF2CC`) avec police orange gras — identifiable d'un coup d'œil
- Feuille *Corrections* : tableau récapitulatif de chaque modification (valeur originale, valeur corrigée, type de correction, justification médicale)
- Généré par `PatientPreprocessExportView` sur GET `/api/patients/preprocess/{id}/export/`

Pandas — Profiling et manipulation des données

Rôle dans le projet : Lit et profile les fichiers Excel/CSV importés : `pd.read_excel()`, `pd.read_csv()` avec détection automatique du séparateur. Normalise les noms de colonnes (`lowercase`, `strip`). Calcule le profil technique : `dtype` par colonne, % valeurs manquantes, nb de doublons, valeurs aberrantes (IQR). Construit les `DataFrame` d'entrée pour le pipeline SVM (`pd.DataFrame([row], columns=feature_keys)`).

SHAP + XGBoost — Sélection des features — analyse post-hoc

Rôle dans le projet : Utilisé en dehors de la plateforme pour justifier la sélection des 24 variables parmi 77 candidates sur HD-478. XGBoost sert de modèle de référence

pour calculer les valeurs SHAP (Shapley Additive exPlanations) : chaque variable reçoit un score d'importance mesurant sa contribution marginale moyenne à la prédiction. Les 24 variables avec les valeurs SHAP les plus élevées (top-24 sur 77) constituent le sous-ensemble sélectionné pour le SVM, auquel s'ajoutent 8 termes d'interaction biologiquement justifiés — soit **32 features** au total.

Chart.js — Visualisation — 12 graphiques interactifs

Rôle dans le projet : Rend les 12 graphiques dynamiques de la plateforme : **Module Patients (6)** : tendance mensuelle des inclusions, camembert sexe, histogramme âge, barres complications, barres étiologies, barres comorbidités. **Module IA (6)** : camembert zones de risque, barres issues prédites, courbe Kaplan-Meier (survie cumulée), barres facteurs contributifs, radar profil de risque, boxplots biologiques par zone. Tous calculés à la volée sur la cohorte active et interactifs (hover, zoom).

Chapitre 1 — Introduction

Figure 1.1 Logo FMPDF

Logotype officiel de la Faculté de Médecine, de Pharmacie et de Dentisterie de Fès (FMPDF), établissement de tutelle académique du projet. Sa présence dans le rapport situe institutionnellement le travail dans le système universitaire marocain (Université Sidi Mohamed Ben Abdellah, Fès).

Figure 1.2 Logo Centre HAIM

Logotype du *Health Artificial Intelligence & Modeling Center* (HAIM), structure de recherche hébergeant le développement de la plateforme. Le HAIM opère en partenariat avec le service de Néphrologie du CHU Hassan II de Fès, qui a fourni les données de la cohorte HD-478 (478 patients hémodialisés, 2020–2024, 95 décès à 1 an, soit un taux de mortalité annuel de 19,9%).

Figure 1.3 Diagramme Kanban Trello — 8 colonnes, 42 tâches

Le diagramme représente le tableau Trello utilisé pour piloter le projet en mode Agile sur 9 sprints de 2 semaines (octobre 2024 – juin 2025). Les 8 colonnes correspondent au cycle de vie complet de chaque tâche : **Backlog** (idées non planifiées), **In Progress** (développement actif), **Review** (relecture code/résultats), **To Validate** (en attente de validation par le superviseur technique M. Zakaria Alami), **Done/Validated** (livrés et intégrés). Les cartes en orange (In Progress) incluent les tâches lourdes : pipeline SVM, intégration ChromaDB, tâches Celery. Les cartes vertes (Done) incluent les 7 services Docker, le schéma PostgreSQL (9 tables), le SPA React (6 pages), le pipeline LLM complet, et le SVM (AUC = 0,826).

Figure 1.4 Stadification IRC selon KDIGO 2012

Grille officielle KDIGO 2012 croisant les 5 stades de DFG_e (G1 : ≥ 90 , G2 : 60–89, G3a : 45–59, G3b : 30–44, G4 : 15–29, G5 : <15 mL/min/1,73m²) avec les 3 catégories d'albuminurie (A1 : <30 , A2 : 30–300, A3 : >300 mg/g). Le code couleur encode le risque composite (vert = faible, jaune = modéré, orange = élevé, rouge = très élevé). Les patients HD-478 sont tous en stade **G5D** (dialyse, DFG_e <15), zone rouge foncé, justifiant la concentration du modèle sur la mortalité à 1 an comme critère primaire.

Chapitre 2 — État de l'art

Ce tableau compare 7 études internationales publiées entre 2016 et 2023, toutes avec le même endpoint (mortalité toutes causes à 1 an en hémodialyse), ce qui rend les AUC-ROC directement comparables. Colonnes : auteurs/année, cohorte (n), pays, algorithmes testés, AUC meilleur, top variables.

Valeurs clés issues des études :

- **Sheng 2020** (PMID 33118948, JMIR Medical Informatics) : $n = 1825$, XGBoost, AUC = 0,843, 42 features candidates, top-15 incluent accès vasculaire, albumine, calcium, PTH, ferritine, cardiopathie, diabète néphropathique. *Référence principale* pour la sélection des 24 variables.
- **Fotheringham 2022** : $n = 56\,000$ (USRDS), AUC = 0,79–0,84.
- **Notre modèle SVM HD-478** : $n = 478$, AUC = 0,826 — dans la médiane haute de l'état de l'art malgré la taille de cohorte 4× plus petite, validant la rigueur du feature engineering.

Chapitre 3 — Vue d'ensemble

Figure 3.1 *Diagramme des cas d'utilisation UML*

Diagramme UML avec 4 acteurs et 12 cas d'utilisation, montrant qui peut faire quoi dans la plateforme.

Les 4 acteurs et leurs droits spécifiques :

- **Super Admin** : tous droits ; créer/supprimer comptes de tous niveaux ; voir tous les AuditLogs ; approuver/rejeter imports ; prédire mortalité.
- **Chef de Service** : comme Super Admin sauf : ne peut pas gérer les comptes Super Admin ; peut approuver les imports.
- **Professeur** : consulter patients, importer fichiers (en attente d'approbation), prédire mortalité, voir ses propres activités.
- **Résident** : comme Professeur ; accès restreint aux statistiques avancées.

Les généralisations (flèches d'héritage) indiquent que chaque acteur hérite des droits du niveau inférieur. Ce diagramme sert de base au RBAC implémenté dans `users/permissions.py`.

Figure 3.2 et Tableau 3.1 *Architecture en cinq couches*

Architecture logicielle à 5 couches superposées :

1. **Présentation** — React 18 SPA, Material-UI, Chart.js, Axios. Communication : HTTP/HTTPS sur port 3000.
2. **Application** — Django REST Framework 4.2, port 8000. Reçoit toutes les requêtes API, applique RBAC, coordonne les composants.
3. **IA/ML** — SVM joblib (prédiction synchrone), Qwen2.5 :14B via Ollama (prétraitement asynchrone via Celery), ChromaDB (RAG embeddings).
4. **Données** — PostgreSQL 16 (port 5432, données permanentes), Redis 7 (port 6379, broker Celery), Filesystem JSON (sessions prétraitement).
5. **Infrastructure** — Docker Compose 7 services, variables d'environnement `.env`, healthchecks entre services.

Le tableau 3.1 précise les protocoles entre couches : REST JSON (1 $\rightarrow$ 2), ORM Django/SQL (2 $\rightarrow$ 4), urllib HTTP (2 $\rightarrow$ 3 Ollama), joblib

(*2eftrightarrow*3 SVM), TCP socket (*2eftrightarrow*4 Redis).

Figure 3.3 *Stack technologique complet*

Vue d'ensemble de **toutes les technologies** regroupées par domaine. Ce diagramme répond à la question “quels outils sont utilisés ensemble?” : Django (backend) + PostgreSQL (données) + Redis (queue) + Celery (async) + Ollama (LLM) + ChromaDB (RAG) + React (UI) + Chart.js (graphiques) + scikit-learn (ML) + Docker (déploiement) — 14 technologies coordonnées dans un seul **docker compose up**.

Chapitre 4 — Pipeline de données

Curation manuelle HD-478

Tableau 4.1 *Dataset HD-478 avant vs. après curation*

Comparaison quantitative du fichier Excel brut hospitalier vs. dataset ML-ready après curation manuelle (9 catégories de corrections) :

Indicateur	Original	Corrigé
Patients	478	478
Variables	160+	160+
Valeurs manquantes	~34%	~18%
Valeurs aberrantes biologiques	287	0
Numéros sériels Excel (dates)	312	0 (dates ISO)
DFGe incohérents	109	0 (recalculés MDRD)
Marqueurs X/x	265	0 (0 ou 1)

La curation a réduit les valeurs manquantes de 34% à 18% et éliminé tous les artéfacts de saisie, permettant l'entraînement d'un modèle ML fiable.

Tableau 4.2 *Conversion dates Excel (312 valeurs)*

Le format Excel stocke les dates comme des entiers (numéros sériels depuis le 01/01/1900). Ex. : le nombre 44927 correspond au 01/01/2023. Sans conversion, Pandas lit ces cellules comme des entiers, rendant impossible tout calcul de survie (délai entre inclusion et décès). La formule appliquée : $date = date_epoch + serial \times 86400s$. Colonnes corrigées : `date_debut_dialyse`, `date_inclusion`, `date_deces` — essentielles pour calculer `delai_jusquau_deces_jours` utilisé par la variable cible du modèle.

Tableau 4.3 *Valeurs biologiques aberrantes (287 corrections)*

287 valeurs biologiques aberrantes détectées par règles cliniques spécifiques : albumine >60 g/L (impossible physiologiquement, max normal 55 g/L) ; créatinine >2000 $\mu\text{mol/L}$;

sodium <100 ou >175 mEq/L ; calcium <1 ou >4 mmol/L ; PTH >5000 pg/mL ; ferritine >10 000 µg/L. Chaque valeur aberrante est remplacée par NaN puis imputée par KNNImputer dans le pipeline ML, ou corrigée manuellement si une explication clinique existe (ex. : unité erronée : albumine en mg/dL au lieu de g/L → divisé par 10).

Tableau 4.4 *Recalcul DFG_e MDRD (109 valeurs)*

Le DFG_e (débit de filtration glomérulaire estimé) de 109 patients était absent ou incohérent. Recalcul par la formule MDRD-4 : $DFG_e = 175 \times [\text{créatinine}_{(\mu\text{mol/L})}/88,4]^{-1,154} \times \text{âge}^{-0,203} \times [0,742 \text{ si femme}]$. Le DFG_e est une variable-clé pour confirmer le stade G5D (DFG_e <15) et n'est pas utilisée directement dans le modèle SVM (évite la fuite de données), mais sert à la validation clinique des dossiers.

Tableau 4.5 *Nettoyage marqueurs X/x — 265 corrections*

Dans les colonnes d'éducation (`niveau_education`) et de statut transplantation, les soignants avaient coché "X" ou "x" au lieu d'écrire 0 ou 1. Python/Pandas interprétait ces cellules comme des `object` (chaîne de caractères), empêchant tout calcul statistique ou ML. Chaque "X"/"x" a été remplacé par la valeur numérique correspondant à la sémantique de la colonne (0 = absent, 1 = présent). 265 corrections réparties sur 8 colonnes.

Tableau 4.6 *Imputation NaN → 0 pour variables transplantation*

Pour 3 variables binaires (`liste_attente_transplantation`, `information_transplantation_donnee`, `transplantation_effectuee`) : les valeurs manquantes signifient cliniquement "non" (absence de mention dans le dossier = événement non survenu). Imputation NaN → 0 appliquée à 187 cellules. Cette règle logique est préférable à l'imputation statistique qui introduirait un biais : imputer 0,3 pour `liste_attente_transplantation` n'a pas de sens clinique.

Tableau 4.7 *Distribution des types de variables (dataset corrigé)*

Après curation, les 160+ variables se répartissent : 68% binaires (0/1, ex. : comorbidités, complications), 24% numériques continues (biologie, durées), 8% catégorielles multi-classes (étiologie, modalité). Cette distribution justifie l'usage de `class_weight=balanced` dans le SVM (les variables binaires ne dominent pas, mais la cible est déséquilibrée 80/20) et le choix de `PowerTransformer` (normalise les distributions asymétriques des variables biologiques continues).

Schéma de base de données

Figure 4.1 *Diagramme ER — schéma complet PostgreSQL*

Diagramme Entité-Relations des **9 tables** de `plateforme_medicale`. Relations clés :
 — `users_utilisateur` → `users_role` (FK, chaque utilisateur a un rôle parmi 4 : `super_admin`, `chef_service`, `professeur`, `résident`)
 — `patients_patient.utilisateur_saisie` est un **CharField** (pas une FK) — stocke

le label de l'utilisateur au moment de la saisie pour préserver l'historique si le compte est supprimé

- `patients_preprocessvalidationrequest` → `users_utilisateur` (`submitted_by` FK + `reviewed_by` FK, nullable)
- `audit_auditlog.action` est un **TextField** (pas **CharField**) pour accueillir des descriptions longues sans troncature
- `audit_hiddenauditlog` → `audit_auditlog` (pour masquer certaines entrées d'audit sans les supprimer)

Tableaux 4.8 à 4.12 Schémas détaillés des tables PostgreSQL

Tableau 4.8 — `users_role` et `users_utilisateur` : `users_role` : id, nom (**CharField** max=50, **UNIQUE**, 4 choix). `users_utilisateur` : hérite **AbstractBaseUser** ; email **UNIQUE** ; nom, prénom, téléphone **CharField** ; `personal_notes` **TextField** ; `role` FK ; `is_active` **BooleanField** ; `date_creation` `auto_now_add`.

Tableau 4.9 — `patients_patient` : 160+ champs ; `icc_charlson` = **CharField**(max_length=10) ; `save()` override auto-génère `id_patient` (format PAT-XXXXXX) ; 11 colonnes **JSONB** pour les sections cliniques ; `extra_data` **JSONB** pour les colonnes dynamiques.

Tableau 4.10 — `patients_preprocessvalidationrequest` : `session_id` **CharField**(64), `status` (pending/approved/rejected), `submitted_by` FK, `reviewed_by` FK (nullable), `quality_score` **FloatField**, `source_file_name`, `submitted_at`, `reviewed_at`, `comment`, `rows_count`, `columns_count`.

Tableau 4.11 — `audit_auditlog` : `action` **TextField**, `entite` **CharField**, `entite_id` **IntegerField**, `details` **TextField**, `adresse_ip` **GenericIPAddressField**, `date` **DateTimeField**, `utilisateur` FK.

Tableau 4.12 — Sections cliniques du modèle Patient : 11 sections couvrent le parcours de soin néphrologique complet : Identité/démographie, Antécédents IRC, Comorbidités, Biologie basale, Dialyse, Complications, Médicaments, Transplantation, Suivi, Outcomes, Données dynamiques (`extra_data` **JSONB**).

Pipeline LLM de prétraitement

Figure 4.2 Pipeline LLM de prétraitement automatisé

Ce diagramme est le plus important du chapitre 4 : il montre comment un fichier Excel brut devient un dataset clinique valide sans saisie manuelle.

11 étapes avec acteurs réels :

1. Clinicien upload le fichier via React : `POST /api/patients/preprocess/analyze/`
2. Django génère un `session_id` **UUID** hex, sauve le fichier dans `/tmp/preprocess/`, crée la session **JSON** (`status`=“pending”)
3. Django retourne `{preprocess_id: "<uuid>"}` au frontend
4. Django dispatche `analyze_preprocess_async.delay()` vers Celery
5. **Celery Worker (asynchrone)** : **Pandas** profile le fichier (nb lignes, colonnes, %

manquants, dtypes, outliers)

6. Celery appelle `_build_retrieval_context()` : ChromaDB (PersistentClient) génère embeddings via `nomic-embed-text` et récupère les normes néphrologiques pertinentes
7. Celery envoie POST `http://ollama:11434/api/generate` avec prompt structuré (profil technique + contexte RAG + échantillons de données)
8. Qwen2.5 :14B retourne un JSON de corrections avec justifications
9. Celery applique 3 passes de correction : plan LLM, KB biologique (normes locales `medical_kb.json`), imputation KNN ($k=5$)
10. Session sauvegardée en JSON filesystem (status="completed")
11. Frontend poll `GET /api/patients/preprocess/{id}/rows/`; clinicien révisé; Chef approuve via `POST /api/patients/preprocess/{id}/integrate/` : INSERT PostgreSQL + AuditLog

Feature Engineering

Tableau 4.14 24 variables cliniques brutes — domaines et littérature

Ce tableau relie chaque variable au modèle SVM à une évidence publiée vérifiée. Groupes :

- **SHAP #1–4 (top confirmés)** : `fistule_arterioveineuse_creee` (accès vasculaire, Sheng 2020 top-15), `couverture_medicale` (accès aux soins, Liyanage 2015 + USRDS 2022), `calcium_basale` (Ca dysrégulé prédit la mortalité, DOPPS : Tentori 2008 + Sheng 2020), `albumine_basale` (chaque -1 g/dL = risque accru, NDT 2005 + Sheng 2020).
- **SHAP top-24** : 9 autres variables confirmées SHAP : événements cardiovasculaires, cathéter tunnellisé, diurèse résiduelle, hospitalisations, PTH, ferritine, séances/semaine, étiologie MRC, année inclusion.
- **Littérature (11 variables)** : HTA, cardiopathie, diabète, dyspnée, œdèmes, douleur abdominale, crise convulsive, maladie héréditaire rénale, hémodialyse, liste attente, information transplantation.

Toutes les 9 références bibliographiques ont été vérifiées sur PubMed.

Tableau 4.15 8 termes d'interaction biologiquement justifiés

Le vecteur de features est étendu de 24 à **32 dimensions** par 8 termes d'interaction :

Interaction	Justification clinique
$\text{age_x_charlson} = \text{âge} \times \text{ICC}$	Risque multiplicatif âge/comorbidités
$\text{dyspnee_x_oedeme} = \text{dyspnée} \times \text{œdèmes}$	Surcharge hydrique synergique
$\text{sodium_lt130} = 1 \text{ si } \text{Na} < 130 \text{ mEq/L}$	Hyponatrémie sévère binaire
$\text{charlson_x_hosp} = \text{ICC} \times \text{hospitalisations}$	Fardeau comorbidités+réadmissions
$\text{hypert_x_cardio} = \text{HTA} \times \text{cardiopathie}$	Risque cardiovasculaire cumulé
$\text{albumine_x_hosp} = \text{albumine} \times \text{hospitalisations}$	Dénutrition + réadmissions
$\text{albumine_lt35} = 1 \text{ si } \text{albumine} < 35 \text{ g/L}$	Seuil clinique dénutrition
$\text{age_x_cardio} = \text{âge} \times \text{cardiopathie}$	Fragilité cardiaque liée à l'âge

Chapitre 5 — Modélisation ML

Comparaison des modèles

Tableau 5.1 Configuration hyperparamètres finaux

Hyperparamètres retenus après `RandomizedSearchCV(n_iter=100, cv=5)` stratifié pour chacun des 9 algorithmes évalués sur HD-478 :

Modèle	Hyperparamètres clés
SVM Linéaire	$C=0,01$, <code>kernel=linear</code> , <code>class_weight=balanced</code>
Régression Log.	$C=0,1$, <code>penalty=l2</code> , <code>solver=lbfgs</code>
Random Forest	<code>n_estimators=200</code> , <code>max_depth=8</code> , <code>min_samples_leaf=3</code>
Extra Trees	<code>n_estimators=300</code> , <code>max_features=sqrt</code>
Gradient Boosting	<code>n_estimators=150</code> , <code>learning_rate=0,05</code> , <code>max_depth=4</code>
AdaBoost	<code>n_estimators=100</code> , <code>learning_rate=0,1</code>
XGBoost	<code>max_depth=4</code> , <code>learning_rate=0,05</code> , <code>subsample=0,8</code>
LightGBM	<code>num_leaves=31</code> , <code>learning_rate=0,05</code>
CatBoost	<code>depth=4</code> , <code>learning_rate=0,05</code> , <code>iterations=500</code>

Tableau 5.2 Comparaison des 9 modèles — résultats HD-478 (10-fold CV)

Résultats complets de la comparaison :

Modèle	AUC-ROC	PR-AUC	F1	Brier
green !10SVM Linéaire	0,826±0,042	0,573	0,512	0,130
LDA	0,823±0,044	0,561	0,498	0,134
Régression Log.	0,821±0,047	0,558	0,491	0,132
Random Forest	0,810±0,055	0,531	0,487	0,148
Gradient Boosting	0,805±0,061	0,519	0,479	0,152
Extra Trees	0,801±0,058	0,515	0,471	0,155
XGBoost	0,809±0,053	0,528	0,483	0,149
AdaBoost	0,791±0,064	0,501	0,462	0,162
CatBoost	0,803±0,057	0,512	0,475	0,153

Le SVM domine sur **toutes les métriques simultanément**.

Figures 5.1–5.2 Courbes ROC — 9 modèles + zoom Top 3

Figure 5.1 — Toutes courbes ROC superposées : axe X = taux faux positifs (1-Spécificité), axe Y = Sensibilité. Diagonale = classifieur aléatoire (AUC=0,5). Le groupe de tête (SVM, LDA, Régression Log.) se détache clairement des modèles ensemblistes qui présentent une courbure plus irrégulière (sur-ajustement sur les petits folds).

Figure 5.2 — Zoom Top 3 : SVM (AUC=0,826, courbe la plus haute dans la région Sensibilité > 0,8), LDA (0,823), Régression Log. (0,821). La différence est visuellement faible : le choix final du SVM est confirmé par les autres métriques (Brier Score, écart train/test, interprétabilité).

Figure 5.3 Barres AUC avec écart-type

Barres horizontales ordonnées : AUC moyen $\pm$ 1 écart-type sur les 10 folds. **SVM** : $0,826 \pm 0,042$ — la barre d'erreur la plus courte parmi les 3 premiers modèles, confirmant sa stabilité. AdaBoost a le plus grand écart ($\pm 0,064$), indicateur d'instabilité aux variations de partition.

Figure 5.4 Écart train-test AUC — détection sur-apprentissage

Pour chaque modèle : AUC_train (mesuré sur l'ensemble d'entraînement de chaque fold) vs. AUC_test. Code couleur : vert si écart < 0,05 (excellent), orange si 0,05–0,10 (acceptable), rouge si > 0,10 (sur-ajustement).

SVM : écart = 0,047 (seul modèle dans la zone verte) ; Random Forest : 0,12 (rouge). Cette figure est décisive : un modèle clinique doit généraliser sur des patients non vus. Un gap élevé (forêts aléatoires) signifie que le modèle a mémorisé l'ensemble d'entraînement sans apprendre les patterns généraux.

Figure 5.5 Radar multi-métriques — 9 modèles

Graphique radar à **5 axes** :

- **AUC-ROC** : discrimination globale
- **PR-AUC** : performance sur la classe minoritaire (décès, 20%)
- **F1-score** : compromis précision/rappel au seuil Youden
- **Brier Score inversé** ($1 - \text{Brier}$) : qualité de calibration

— **Recall** : fraction de décès détectés (métrique critique en clinique)

Le polygone SVM est le plus **équilibré** et le plus **étendu**. Les forêts aléatoires ont un bon Recall mais un Brier Score élevé (probabilités mal calibrées). Un bon Recall sans bonne calibration est inutile cliniquement : le médecin a besoin de probabilités fiables, pas seulement de classifications.

Figure 5.6 Brier Score par modèle

Le Brier Score mesure l'erreur quadratique moyenne entre la probabilité prédite $\hat{p}$ et la réalité $y \in \{0, 1\}$: $BS = \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i)^2$. Score parfait = 0, aléatoire = 0,25 (prévalence 50%).

Seuil excellent : BS < 0,15 (ligne pointillée sur le graphe). SVM = 0,130 et Régression Log. = 0,132 passent ce seuil, les autres non. Cliniquement : si le SVM prédit 65% de risque, environ 65% des patients dans ce décile mourront réellement dans l'année — les probabilités sont utilisables pour prioriser les soins.

Figure 5.7 Diagramme de fiabilité (calibration)

Reliability plot pour les 3 meilleurs modèles. Les probabilités prédites sont groupées en déciles (axe X : 0–0,1, 0,1–0,2, ..., 0,9–1,0). Pour chaque décile, la fréquence de décès observée est calculée (axe Y). La diagonale parfaite signifie : “si je prédis 30% de risque, 30% de ces patients décèdent effectivement”.

SVM + Platt Scaling : courbe la plus proche de la diagonale ; l'écart moyen (*calibration error*) est minimal. Régression Log. : bien calibrée aussi mais légèrement optimiste à fort risque. Random Forest : nettement sous-calibré aux probabilités élevées (prédit 80% mais seulement 60% de décès observés) — dangereux cliniquement car surestime la confiance.

Figures 5.8 à 5.10 Matrices de confusion — SVM, LDA, Régression Log.

Figure 5.8 — SVM Linéaire (seuil Youden = 0,238) : La matrice de confusion présente 4 quadrants : VP (vrais positifs, décès correctement détectés), FN (faux négatifs, décès manqués), FP (fausses alarmes), VN (vrais négatifs, survivants corrects). La disposition est : 95 décès réels (VP + FN) en ligne haute, 383 survivants réels (FP + VN) en ligne basse.

Sensibilité (vrais positifs / tous positifs réels) : fraction des décès correctement détectés ; Spécificité : fraction des survivants correctement classés. La pondération `class_weight=balanced` maximise la Sensibilité sur la classe minoritaire (décès, 20%). Le seuil de Youden (0,238, inférieur au seuil par défaut de 0,5) reflète la nécessité de détecter plus de décès au prix de quelques fausses alarmes.

Figure 5.9 — LDA (seuil 0,220) : performances proches du SVM mais Brier Score légèrement supérieur ; non sélectionné en raison de son hypothèse de normalité multivariée non vérifiable sur des données réelles mixtes (binaires + continues).

Figure 5.10 — Régression Log. (seuil 0,502) : seuil plus élevé car distribution de probabilités mieux centrée sur 0,5 ; bon Brier Score mais moins stable sur les folds (écart-type AUC = 0,047 vs 0,042 SVM).

Figure 5.11 *Courbes Précision-Rappel*

Les courbes PR sont la **métrique principale en présence de déséquilibre de classes** (80% survivants / 20% décès). L'axe X est le Rappel (fraction de décès détectés), l'axe Y la Précision (fraction de vraies alarmes parmi les alertes émises). La ligne de base = prévalence = 0,20 (classifieur aléatoire).

PR-AUC SVM = 0,573 (vs. 0,20 pour le hasard) : le modèle a une capacité de détection effective 2,9× supérieure au hasard. Cette figure est présentée aux cliniciens pour expliquer le compromis : augmenter le Rappel (détecter plus de décès) diminue la Précision (plus de fausses alarmes).

Figure 5.12 *Boxplot AUC sur 10 folds*

Boîtes à moustaches : médiane (trait central), Q1-Q3 (boîte), min-max (moustaches). Ce graphe répond à : “est-ce que le modèle est stable ou chanceux ?”

SVM : boîte étroite centrée sur 0,826 — aucun fold avec AUC < 0,78 ; AdaBoost : boîte large, un fold avec AUC < 0,70 (inacceptable en clinique). La stabilité inter-folds est un critère de confiance clinique : un modèle qui varie de 0,73 à 0,89 selon le fold est imprévisible en production.

Zones de risque

Tableau 5.3 *Classification par zones de risque — seuils HD-478*

Les seuils T_1 et T_2 sont dérivés par **bootstrap ROC/Youden** ($n = 1000$ itérations) sur HD-478 et stockés dans `predictions/models/seuils_classification.joblib`. Chargés dynamiquement par `_resolve_thresholds(metadata)` avec priorité : (1) `seuils_classification.joblib`, (2) `calibration_thresholds`, (3) `gmm_thresholds`, (4) défaut fixe 0,10/0,29.

Taux de mortalité observés dans la cohorte HD-478 par zone :

Zone	Seuil	Mortalité observée	Patients
Faible	$\hat{p} < T_1$	3,8%	~240
Modérée	$T_1 \leq \hat{p} < T_2$	15,6%	~155
Élevée	$\hat{p} \geq T_2$	46,5%	~83

La séparation est significative (log-rank $p < 0,001$) : les 3 zones captent 3 populations de survie biologiquement distinctes.

Figure 5.13 *Workflow complet de prédiction (diagramme de flux)*

14 étapes représentées avec les acteurs réels du code :

1. Clic “Prédire” sur l’interface React
2. GET `/api/predictions/patient/{id}/mortalite/`
3. `Patient.objects.get(pk=patient_id)` o PostgreSQL
4. Objet ORM retourné

5. `svm_extract_features(patient)` o dict de 32 features
6. `pd.DataFrame([row], columns=feature_keys)`
7. `pipeline.predict_proba(input_df)[: ,1][0]` o $\hat{p}$ calibrée Platt (1 seul appel)
8. `_resolve_thresholds(metadata)` o T_1, T_2 chargés depuis joblib
9. Classification : Faible / Modéré / Élevé
10. `_build_svm_factors(pipeline, feature_keys)` o top-5 coefficients
11. `_save_last_prediction(result)` o JSON filesystem (pas PostgreSQL)
12. Réponse : `{niveau_risque, score_risque, probabilite_deces, factors, recommendation, mort_rates}`
13. Rendu jauge + facteurs dans React

Chapitre 6 — Architecture logicielle

Figures 6.1–6.2 *Structure des applications Django*

Figure 6.1 : arborescence des 5 apps Django : `users/` (models, serializers, views, permissions, urls), `patients/` (models 160+ champs, views 3800+ lignes, tasks Celery, preprocess_rag), `predictions/` (views, predict_mortalite, models/feature_mapping, train_from_excel), `audit/` (models, views), `config/` (settings, urls, celery).

Figure 6.2 : chaque app suit le patron DRF : Modèle (ORM Django) → Sérialiseur (validation + transformation JSON) → Vue (logique métier APIView) → URL (routage). La séparation garantit que modifier la validation d'un champ (sérialiseur) n'affecte pas la logique métier (vue), et vice versa.

Figure 6.3 *Diagramme de classes UML — modèles Django*

Classes représentées avec leurs attributs réels (types vérifiés dans le code) :

- **Role** : nom = CharField(max=50, choices, unique=True)
- **Utilisateur** : email(unique), nom, prenom, telephone CharField; personal_notes TextField; role FK; hérite AbstractBaseUser
- **Patient** : icc_charlson = **CharField** (pas Integer); utilisateur_saisie = **CharField** (pas FK); save() auto-assign id_patient; `extract_features_for_patient()` est dans `predictions/models/feature_mapping.py` (pas une méthode Patient)
- **AuditLog** : action = **TextField** (pas CharField)
- Relation Patient-Utilisateur : **association simple** via CharField (pas agrégation, car la référence est un label texte figé)

Tableau 6.1 *Endpoints REST principaux*

40+ endpoints groupés par ressource. Les plus utilisés dans le projet :

Méthode + URL	Vue	Droits
POST /api/auth/login/	LoginView	Public
POST /api/auth/token/refresh/	TokenRefreshView	Public
POST /api/patients/preprocess/analyze/	PatientPreprocessAnalyzeView	Auth
GET /api/patients/preprocess/{id}/rows/	PatientPreprocessRowsView	Auth
POST /api/patients/preprocess/{id}/integrate/	PatientPreprocessIntegrateView	Chef
GET /api/predictions/patient/{id}/mortalite/	PredictMortalitePatientView	Auth

Figure 6.4 *Flux JWT — authentification et rafraîchissement*

Diagramme de séquence à 3 acteurs (Browser, Django API, PostgreSQL) en 5 étapes :

1. POST /api/auth/login/ body : {email, password}
2. SELECT users_utilisateur WHERE email=? o hachage bcrypt vérifié
3. Émission : access JWT (8h, payload custom) + refresh JWT (1j)
4. Requêtes authentifiées : header Authorization: Bearer <access>; verification HMAC-SHA256 côté Django
5. Refresh : POST /api/auth/token/refresh/ + refresh token o nouveau access JWT (8h)

Important : l'intercepteur Axios ajoute uniquement le Bearer token aux requêtes sortantes ; il n'y a **pas** de refresh automatique sur expiration (pas d'intercepteur RESPONSE sur 401 dans le code actuel).

Tableau 6.2 *Matrice de permissions RBAC*

Croise 4 rôles × 12 actions. Exemples des règles les plus importantes : **Super Admin** : seul à pouvoir supprimer des comptes Super Admin et voir tous les AuditLogs (y compris masqués). **Chef de Service** : peut approuver/rejeter les imports et voir les AuditLogs mais pas ceux masqués. **Professeur/Résident** : peuvent importer et prédire mais voient uniquement leur propre fil d'activité (pas les AuditLogs globaux). Implémentée dans users/permissions.py : classes IsSuperAdmin et IsAdminOrChefService, appliquées via permission_classes = [IsAdminOrChefService] dans chaque vue.

Figures 6.5 à 6.36 *Captures d'écran de l'interface (30 figures)*

Landing page (6.5) : présente les 3 modules (gestion patients, IA prétraitement, prédiction ML), accessible sans authentification.

Login (6.6) : split-panel. Côté droit : formulaire JWT. Après connexion réussie : jwtDecode(access) extrait id, email, nom, rôle ; navigate('/dashboard') redirige.

Sidebar (6.7) : items de navigation filtrés selon user.role via ProtectedRoute.

Dashboard Super Admin (6.9–6.12) : 7 KPI cards interactives ; création/modification/suppression de comptes avec mot de passe de confirmation obligatoire (implémenté dans la vue DRF : confirm_admin_password(request)).

Prétraitement (6.18–6.25) : interface complète du pipeline LLM : upload o barre de progression temps réel (polling toutes les 2s) o tableau de révision cellule par cellule o export Excel coloré o intégration PostgreSQL.

Simulation clinique (6.34) : le clinicien modifie les valeurs des 32 features et voit instantanément la nouvelle prédiction via `POST /api/predictions/patient/{id}/mortalite/` (mode simulation).

Résultat prédiction (6.35) : jauge semi-circulaire (zone + $\hat{p}$), top-5 facteurs (coefficients SVM), texte de recommandation personnalisé avec taux de mortalité observé dans la zone correspondante sur HD-478.

Monitor (6.36) : chronologie longitudinale par patient : évolution des prédictions, modifications de champs, actions des soignants — alimenté par les entrées AuditLog.

Figures 6.37 à 6.48 *Graphiques analytiques et diagrammes de séquence*

Graphiques analytiques (6.37–6.48, Chart.js) :

- **6.37 — Tendance mensuelle** : nb inclusions par mois 2020–2024, révèle les pics saisonniers et la croissance de la cohorte.
- **6.38 — Sexe + KPIs** : donut Homme/Femme + 4 cartes : âge médian, albumine moyenne, durée dialyse médiane, taux mortalité 1 an.
- **6.43 — Zones de risque** : camembert Faible/Modéré/Élevé avec taux observés (3,8% / 15,6% / 46,5%).
- **6.45 — Kaplan-Meier** : 3 courbes de survie par zone, log-rank $p < 0,001$.
- **6.46 — Facteurs contributifs** : top-10 coefficients SVM ordonnés par valeur absolue — base de l'interprétabilité clinique.
- **6.47 — Radar profil de risque** : profil patient (rouge) vs. médiane cohorte (bleu) sur 8 dimensions.

Diagrammes de séquence (6.48–6.50) : Voir les figures détaillées dans les sections Chapitres précédentes. Les 3 séquences couvrent : authentification JWT (5 étapes, 3 acteurs), import LLM (18 étapes, 7 acteurs, Celery asynchrone), prédiction SVM (11 étapes, 5 acteurs, 1 seul appel `predict_proba`).

Résumé : Ce guide documente **50+ figures** et **14 outils/technologies** avec leurs rôles précis dans le projet. Chaque valeur numérique citée (AUC, seuils, paramètres, ports) est conforme au code source du projet. Chaque référence bibliographique a été vérifiée sur Pub-Med.